S2D-08: Dashboard Migration Best Practices

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 11/02/2026

Overview

This notebook provides guidance for migrating Splunk dashboards to Dynatrace. While there's no automated conversion tool, understanding the mapping between Splunk and Dynatrace visualization concepts enables efficient manual migration.

Table of Contents

1. Dashboard Structure Comparison
2. Visualization Type Mapping
3. Query Translation Examples
4. Using Variables for Filtering
5. Dashboard Organization Best Practices
6. Log Searcher Dashboard Pattern
7. Migration Checklist

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Dashboards
Permissions	`document:documents:write`, `storage:logs:read`, `storage:buckets:read`
Knowledge	Completed SPL to DQL translation (S2D-03)

Learning Objectives

By the end of this notebook, you will be able to:

1. Map Splunk visualization types to Dynatrace equivalents
2. Convert Splunk dashboard structure to Dynatrace format
3. Apply best practices for dashboard organization
4. Use variables for interactive filtering

Dashboard Structure Comparison

Splunk Dashboard Components

Component	Purpose
Dashboard	Container for panels
Panel	Individual visualization
Search	SPL query
Input	User filter (dropdown, text)
Drilldown	Navigation action

Dynatrace Dashboard Components

Component	Purpose
Dashboard	Container for tiles
Tile	Individual visualization
DQL Query	Data source
Variable	User filter
Drilldown	Navigation action

Visualization Type Mapping

Direct Equivalents

Splunk	Dynatrace	Notes
Single Value	Single Value	Key metric display
Line Chart	Line Chart	Timeseries visualization
Bar Chart	Bar Chart	Category comparison
Pie Chart	Pie Chart	Distribution
Table	Table	Tabular data
Area Chart	Area Chart	Stacked timeseries
Column Chart	Bar chart or Categorical chart	Vertical bars

Splunk-Specific Visualizations

Splunk	Dynatrace Alternative
Choropleth Map	Choropleth map
Radial Gauge	Gauge
Filler Gauge	Meter bar or Gauge

Splunk	Dynatrace Alternative
Marker Gauge	Meter bar or Gauge
Scatter Chart	Scatterplot

The Dynatrace visualization list for dashboards and notebooks includes Gauge, Meter bar, Choropleth map, Dot map and Scatterplot.

Query Translation Examples

Single Value Panel

```
// Single value: Total error count
// Splunk: index=app level=ERROR | stats count
fetch logs, from:-1h
| filter status == "ERROR"
| summarize total_errors = count()
```

Time-Series Line Chart

```
// Line chart: Error trend over time
// Splunk: index=app level=ERROR | timechart count
fetch logs, from:-24h
| filter status == "ERROR"
| makeTimeseries error_count = count(), interval:5m
```

Bar Chart by Category

```
// Bar chart: Errors by service
// Splunk: index=app level=ERROR | stats count by service
fetch logs, from:-1h
| filter status == "ERROR"
| summarize error_count = count(), by:{k8s.deployment.name}
| sort error_count desc
| limit 10
```

Table with Multiple Columns

```
// Table: Service health summary
// Splunk: index=app | stats count, count(eval(level="ERROR")) as errors by
service
fetch logs, from:-1h
| summarize {
    total = count(),
    errors = countIf(status == "ERROR"),
    warnings = countIf(loglevel == "WARN")
  }, by:{k8s.deployment.name}
| fieldsAdd error_rate = round((toDouble(errors) / toDouble(total)) * 100,
decimals: 2)
| sort errors desc
```

Using Variables for Filtering

Defining Variables

Dynatrace dashboards support variables for interactive filtering:

Variable Type	What it is
DQL	*"the value is returned from a query you enter when you define the variable."*
Code	*"the value is returned from code you enter when you define the variable."*
List	*"a comma-separated values (CSV) list of values."*
Free Text	*"free text. You can enter a Default value."*

> **Sources:** Variable component (DT docs).

Using Variables in Queries

```
// Using dashboard variables in DQL
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
fetch logs, from:-1h
// dashboard tile: | filter matchesPhrase(k8s.cluster.name, $cluster)
| filter isNotNull(k8s.cluster.name)
// dashboard tile: | filter matchesPhrase(k8s.namespace.name, $namespace)
| filter isNotNull(k8s.namespace.name)
| summarize count = count(), by:{loglevel}
```

Variable-Populated Dropdown Query

```
// Query to populate a deployment dropdown variable
fetch logs, from:-1h
| filter isNotNull(k8s.deployment.name)
| summarize count(), by:{k8s.deployment.name}
| fields k8s.deployment.name
| sort k8s.deployment.name asc
```

Dashboard Organization Best Practices

Layout Recommendations

1. **Top Row: Key Metrics**
   - Single value tiles showing critical KPIs
   - Use color thresholds for status indication
2. **Middle Section: Trends**
   - Time-series charts showing patterns over time
   - Use consistent time intervals across related charts
3. **Bottom Section: Details**
   - Tables with detailed breakdown

- Log samples for investigation

Naming Conventions

Follow the naming standards (see S2D-09):

- Dashboard: `[AppName] Dashboard Title`
- Example: `[EasyTravel] Business Overview`

Log Searcher Dashboard Pattern

A common migration pattern is creating log searcher dashboards for investigating logs.

VM Log Searcher

Variables: `host_name`, `log_source`

```
// VM Log Searcher - Log count by host
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
fetch logs, from:-1h
// dashboard tile: | filter matchesPhrase(host.name, $host_name)
| filter isNotNull(host.name)
// dashboard tile: | filter matchesPhrase(log.source, $log_source)
| filter isNotNull(log.source)
| summarize count = count(), by:{host.name}
| sort count desc
| limit 20
```

```
// VM Log Searcher - Log count by source
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
fetch logs, from:-1h
// dashboard tile: | filter matchesPhrase(host.name, $host_name)
| filter isNotNull(host.name)
// dashboard tile: | filter matchesPhrase(log.source, $log_source)
| filter isNotNull(log.source)
| summarize count = count(), by:{host.name, log.source}
| sort count desc
| limit 20
```

Kubernetes Log Searcher

Variables: `k8s_cluster`, `k8s_namespace`, `k8s_deployment`

```
// K8s Log Searcher - Log count by cluster
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
fetch logs, from:-1h
// dashboard tile: | filter matchesPhrase(k8s.cluster.name, $k8s_cluster)
| filter isNotNull(k8s.cluster.name)
| summarize count = count(), by:{k8s.cluster.name}
| sort count desc
```

```
// K8s Log Searcher - Log count by deployment
//
// Corrected 08/12/2026: `$variable` references resolve only inside a
Dashboard tile — executed as a
// notebook cell they fail at parse time with "`$` isn't allowed here". The
literals below make the
// cell runnable; the commented line above each filter is the form to paste
into a dashboard tile.
fetch logs, from:-1h
// dashboard tile: | filter matchesPhrase(k8s.deployment.name,
$k8s_deployment)
| filter isNotNull(k8s.deployment.name)
| summarize count = count(), by:{k8s.cluster.name, k8s.namespace.name,
k8s.deployment.name}
| sort count desc
| limit 20
```

Migration Checklist

Step	Action
1	Inventory all Splunk dashboard panels
2	Translate SPL queries to DQL (S2D-03)
3	Create Dynatrace dashboard with appropriate layout
4	Add tiles with translated queries
5	Configure variables for filtering
6	Apply naming conventions
7	Validate data matches Splunk
8	Share with stakeholders

Next Steps

- **S2D-09: Naming Standards** - Apply consistent naming conventions

References

- Dynatrace Dashboards
- Dashboards and notebooks (DT docs)
- Edit visualizations (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.